

ALGORITMI DI ORDINAMENTO

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



INTRODUZIONE

■ Problema dell'ordinamento

■ **Input:** una sequenza di n numeri $[a_1, a_2, \dots, a_n]$

■ **Output:** una permutazione $p : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ degli indici

$$[a_{p(1)}, a_{p(2)}, \dots, a_{p(n)}]$$

tale che

$$a_{p(1)} \leq a_{p(2)} \leq \dots \leq a_{p(n)}$$

■ Esempio:

■ Input: $[7, 32, 88, 21, 92, -4]$, Output: $[-4, 7, 21, 32, 88, 92]$

■ In questa lezione discutiamo alcuni algoritmi di ordinamento

■ **Incrementali:** SelectionSort, InsertionSort

■ **Divide et Impera:** MergeSort, QuickSort

■ **Non-comparativi:** CountingSort, RadixSort

■ Discuteremo, inoltre, anche il **limite inferiore alla complessità del problema dell'ordinamento**

NOZIONI PRELIMINARI

- Assumiamo di avere in input un **array** di cui ogni elemento ha
 - una **chiave**, tutte le chiavi sono confrontabili rispetto a $<$, $=$, $>$
 - un **valore**, contenuto associato alla chiave
- Ordiniamo l'array rispetto alla chiave, non rispetto al valore
 - In molti casi il nostro array conterrà solo la chiave
- Definiamo due proprietà degli algoritmi di ordinamento
 - **in place**: l'algoritmo riordina gli elementi nell'array in input (non è richiesto nessun array aggiuntivo per il calcolo)
 - **stabile**: valori con la stessa chiave appaiono nell'array ordinato nello stesso ordine in cui appaiono nell'array in input
- Nota: algoritmi non stabili possono sempre essere trasformati in stabili
 - Usiamo come chiave la coppia (*key*, *index*)
 - *index* è la posizione del valore nell'array in input
 - *key* è la chiave primaria di ordinamento, *index* la secondaria
 - Esempio: $(10, 2) < (10, 6)$

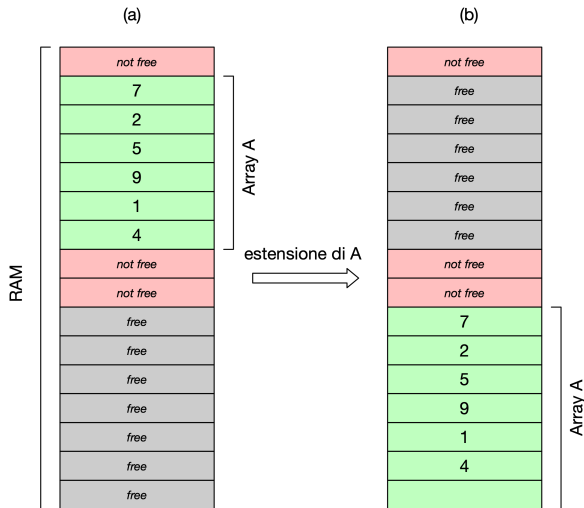
LA STRUTTURA DATI ARRAY

- **Array**: struttura dati costituita da una sequenza di **elementi omogenei** accessibili tramite un indice (primo indice=1 nello pseudocodice)

	1	2	3	4	5	6
A	7	2	5	9	1	4

- E' possibile accedere agli elementi dell'array tramite il loro indice
 - Esempio: $A[1] = 7$, $A[5] = 1$
 - Il costo di accesso è **costante** per tutti gli elementi
- Usiamo la notazione $A[i, \dots, j]$ per indicare il sotto-array di elementi $A[i], A[i + 1], \dots, A[j]$
- Gli array sono una struttura dati a **dimensione fissa**
 - La dimensione è fissata quando l'array viene creato
 - Un array può essere ridimensionato (ampliato) solo creando un nuovo array della dimensione desiderata e copiando tutti gli elementi dal vecchio al nuovo array

RIDIMENSIONAMENTO DI UN ARRAY



- (a) Non c'è spazio libero contiguo in RAM per estendere A
- (b) Estendiamo A spostandolo in una nuova posizione in RAM

ALGORITMI DI ORDINAMENTO INCREMENTALI

- Algoritmi semplici che costruiscono l'ordinamento un elemento alla volta
- Idea: partendo da un **prefisso ordinato** $A[1, \dots, k]$ dell'intero array **estendiamo** l'ordinamento ad un prefisso più ampio $A[1, \dots, k + 1]$
- **SelectionSort**
 - Cerca il minimo in $A[k + 1, \dots, n]$ e lo sposta in posizione $k + 1$
- **InsertionSort**
 - Inserisce l'elemento $A[k + 1]$ nella corretta posizione in $A[1, \dots, k + 1]$
- Entrambi gli algoritmi sono **in place**

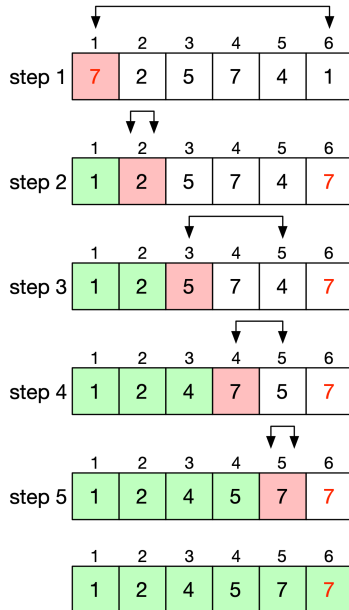
SELECTIONSORT (ANNI 50?)

■ Ad ogni passo $i = 1, \dots, n - 1$

1 cerca la posizione j della minima chiave in $A[i, \dots, n]$

2 scambia (swap) $A[j]$ con $A[i]$

■ Algoritmo non stabile



SELECTIONSORT: PSEUDOCODICE

```
1: function SELECTIONSORT(INT  $A[1, \dots, n]$ )
2:   for  $i = 1, \dots, n - 1$  do
3:      $m = \text{MIN}(A, i, n)$  ▷ Search min in  $A[i, \dots, n]$ 
4:     ▷ Swap  $A[m]$  with  $A[i]$ 
5:     if  $m \neq i$  then
6:       SWAP( $A, i, m$ )
7:   function MIN(INT  $A[1, \dots, n]$ , INT  $i$ )
8:     for  $j = i + 1, \dots, n$  do
9:       if  $A[j] < A[i]$  then
10:         $i = j$ 
11:   return  $i$ 
12: function SWAP(INT  $A[1, \dots, n]$ , INT  $i$ , INT  $j$ )
13:    $tmp = A[i]$ 
14:    $A[i] = A[j]$ 
15:    $A[j] = tmp$ 
```

- La funzione SWAP scambia due elementi nell'array
- Lo SWAP a riga 6 rende l'algoritmo non stabile

COMPLESSITÀ COMPUTAZIONALE DI SELECTIONSORT

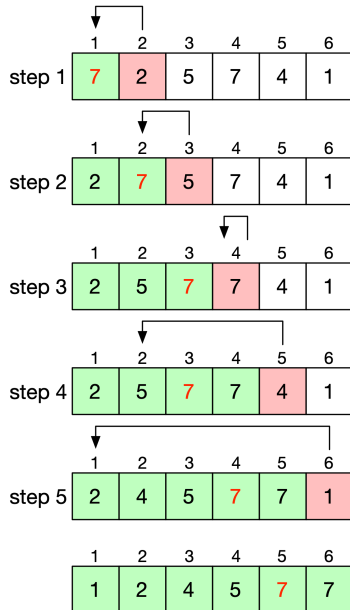
- Il ciclo a riga 2 viene eseguito $n - 1$ volte. Al passo i
 - Costo **ricerca del minimo** (riga 3): $\Theta(n - i)$ ($n - i$ iterazioni)
 - Costo **swap** (riga 6): $O(1)$ (operazioni costanti)
- I due cicli sono eseguiti interamente ogni volta
 - Il caso ottimo, medio e pessimo coincidono
- Costo pessimo, medio e ottimo:

$$(n - 1) + (n - 2) + \cdots + 1 = \sum_{i=1}^{n-1} n - i = \sum_{i=1}^{n-1} i = \frac{n(n - 1)}{2} = \Theta(n^2)$$

- **Complessità quadratica** sulla lunghezza dell'array

INSERTIONSORT (JOHN MAUCHLY, 1946?)

- Ad ogni passo $i = 2, \dots, n$
 - 1 $A[1, \dots, i - 1]$ è ordinato
 - 2 inserisci $A[i]$ nella posizione corretta in $A[1, \dots, i]$
- Intuitivamente, approccio utilizzato per ordinare carte da gioco
- Algoritmo stabile



INSERTIONSORT: PSEUDOCODICE

```
1: function INSERTIONSORT(INT  $A[1, \dots, n]$ )  
2:   for  $i = 2, \dots, n$  do  
3:      $j = i$   
4:     while  $j > 1$  and  $A[j] < A[j - 1]$  do  
5:       SWAP( $A, j, j - 1$ )  
6:        $j = j - 1$ 
```

■ Notiamo che:

- il ciclo for a riga 2 viene eseguito sempre interamente
- il ciclo while a riga 4 potrebbe non essere eseguito

COMPLESSITÀ COMPUTAZIONALE DI INSERTIONSORT

■ Costo nel caso pessimo

- Le chiavi nell'array sono ordinate dalla più grande alla più piccola
- Il ciclo for a riga 2 viene eseguito $n - 1$ volte
- Per ogni iterazione i del ciclo for, il ciclo while a riga 4 viene eseguito $i - 1$ volte

$$\sum_{i=2}^n (i - 1) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$$

■ Costo nel caso ottimo

- Le chiavi nell'array sono ordinate dalla più piccola alla più grande
- Il ciclo for a riga 2 viene eseguito $n - 1$ volte
- Il ciclo while a riga 4 non viene mai eseguito ($\forall j, A[j] \geq A[j - 1]$)

$$\sum_{i=2}^n 1 = n - 1 = \Theta(n)$$

- Complessità quadratica nel caso pessimo e lineare nel caso ottimo

COMPLESSITÀ DI INSERTIONSORT NEL CASO OTTIMO

- Il caso ottimo si verifica anche per array **quasi ordinati**
- Esempio: A già ordinato tranne che per le k chiavi più piccole
 - Il ciclo while (riga 5) sarà eseguito solo su tali k elementi
 - Al peggio, ogni volta il ciclo while verrà eseguito per n iterazioni
 - Il costo per ordinare tale array quasi ordinato è quindi

$$\Theta(n) \text{ (ciclo for)} + O(nk) \text{ (ciclo while)} = O(nk)$$

- Quando il numero k di elementi non ordinati è costante rispetto ad n , cioè $k = O(1)$, il ciclo while ha costo $O(n)$ e quindi InsertionSort ha un costo complessivo pari a $\Theta(n)$ (**costo lineare**)
- Questa proprietà vale anche per altri casi di array quasi ordinati
 - Ad esempio, vale per array ordinati tranne che per $k = O(1)$ chiavi
 - Più complesso da analizzare rispetto al caso " k chiavi più piccole "
- Array quasi ordinati sono abbastanza comuni in alcune applicazioni reali

COMPLESSITÀ DI INSERTIONSORT NEL CASO MEDIO

- Dobbiamo formulare qualche assunzione probabilistica per il caso medio
- Ricordiamo che:
 - Il ciclo for (riga 2) viene sempre eseguito interamente
 - Ad ogni iterazione i del ciclo for, il ciclo while viene eseguito al massimo per $i - 1$ iterazioni (da 0 a $i - 1$ iterazioni)
 - Possiamo solo fare assunzioni sulla lunghezza media del ciclo while
- Assunzione ragionevole: ciclo while eseguito in media per $(i - 1)/2$ volte
- Sotto tale assunzione probabilistica abbiamo il costo totale

$$\sum_{i=2}^n \frac{i-1}{2} = \frac{1}{2} \sum_{i=1}^{n-1} i = \frac{1}{2} \frac{n(n-1)}{2} = \Theta(n^2)$$

- Il **costo medio** di InsertionSort è **quadratico**, come nel caso pessimo

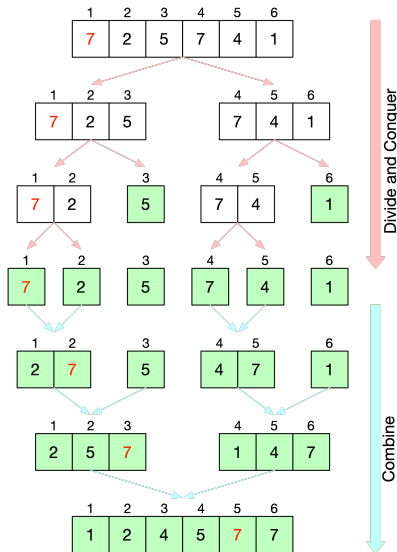
ALGORITMI DI ORDINAMENTO DIVIDE ET IMPERA

- **Divide et impera**. Strategia per ottenere controllo militare e politico: fare in modo che gli avversari siano divisi e si combattano tra di loro
 - Denominata anche **Divide and Conquer** in inglese
- Divide et impera negli algoritmi:
 - **Divide**: dividere il problema in sotto-problemi più piccoli
 - **Conquer**: risolvere ("conquistare") i sotto-problemi ricorsivamente
 - **Combine**: fondere le soluzioni ai sotto-problemi in una più generale
- Vediamo due algoritmi di ordinamento nella classe divide et impera
 - MergeSort
 - QuickSort
- Tipicamente più efficienti degli algoritmi incrementali

MERGESORT (JOHN VON NEUMANN, 1945)

- 1 Divide:** dividiamo $A[1 \dots n]$ in due metà $A_1 = A[1, \dots, q]$ e $A_2 = A[q+1, \dots, n]$ dove $q = \lfloor (1+n)/2 \rfloor$
- 2 Conquer:** richiamiamo ricorsivamente l'algoritmo su A_1 and A_2 se hanno lunghezza > 1 (un array di lunghezza 1 è già ordinato)
- 3 Combine:** combiniamo i due array ordinati A_1, A_2 in un unico array ordinato

Algoritmo stabile



MERGESORT: PSEUDOCODICE

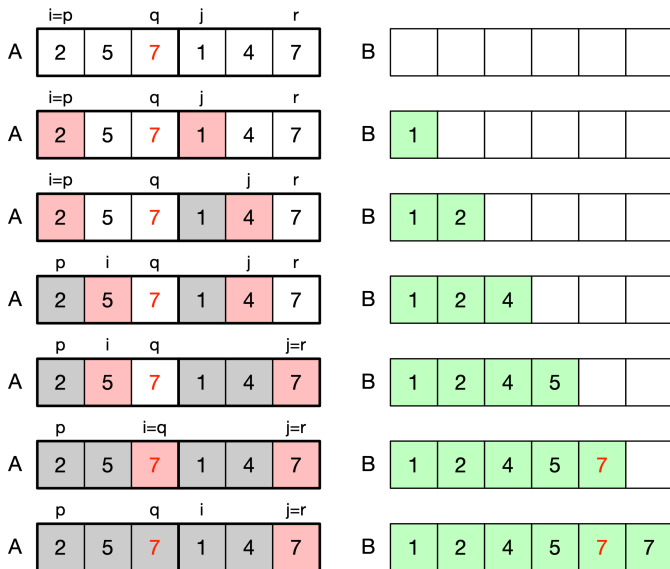
```
1: function MERGESORT(INT  $A[1, \dots, n]$ , INT  $p$ , INT  $r$ )
2:   if  $p < r$  then
3:      $q = \lfloor (p + r) / 2 \rfloor$ 
4:     MERGESORT( $A, p, q$ )
5:     MERGESORT( $A, q + 1, r$ )
6:      $\triangleright$  Here  $A[p, \dots, q]$  and  $A[q + 1, \dots, r]$  are sorted
7:     MERGE( $A, p, q, r$ )
```

- Prima chiamata: $\text{MERGESORT}(A, 1, n)$
- E' semplice dividere, più complesso ricombinare
- La funzione MERGE combina gli array ordinati $A[p, \dots, q]$, $A[q+1, \dots, r]$
- Dopo MERGE il sotto-array $A[p, \dots, r]$ è ordinato
- MERGE può essere implementata in modo che MERGESORT sia stabile
- Complesso implementare MERGE in modo che MERGESORT sia in-place

MERGESORT: PSEUDOCODICE DI MERGE

```
1: function MERGE(INT  $A[1, \dots, n]$ , INT  $p$ , INT  $q$ , INT  $r$ )
2:   LET  $B[1, \dots, r - p + 1]$  BE A NEW ARRAY
3:    $i = p$   $\triangleright$  Index over  $A[p, \dots, q]$ 
4:    $j = q + 1$   $\triangleright$  Index over  $A[q + 1, \dots, r]$ 
5:    $k = 1$   $\triangleright$  Index over  $B[1, \dots, r - p + 1]$ 
6:   while  $i \leq q$  and  $j \leq r$  do
7:     if  $A[i] \leq A[j]$  then  $\triangleright$  Condition  $A[i] \leq A[j]$  makes the algorithm stable
8:        $B[k] = A[i]$ 
9:        $i = i + 1$ 
10:    else
11:       $B[k] = A[j]$ 
12:       $j = j + 1$ 
13:     $k = k + 1$ 
14:     $\triangleright$  If  $i \leq q$  append  $A[i, \dots, q]$  to  $B[1, \dots, k - 1]$ 
15:    while  $i \leq q$  do
16:       $B[k] = A[i]$ 
17:       $k = k + 1, i = i + 1$ 
18:     $\triangleright$  If  $j \leq r$  append  $A[j, \dots, r]$  to  $B[1, \dots, k - 1]$ 
19:    while  $j \leq r$  do
20:       $B[k] = A[j]$ 
21:       $k = k + 1, j = j + 1$ 
22:     $\triangleright$  Copy the sorted numbers from  $B[1, \dots, r - p + 1]$  to  $A[p, \dots, r]$ 
23:    for  $k = 1, \dots, r - p + 1$  do
24:       $A[p + k - 1] = B[k]$ 
```

MERGESORT: ESECUZIONE DI MERGE



Alla fine ricopiamo B in A

COMPLESSITÀ COMPUTAZIONALE DI MERGESORT

- La procedura MERGE ha un costo lineare sulla lunghezza $r - p + 1$
 - In ognuno dei tre cicli while incrementiamo i oppure j , mai insieme
 - i viene incrementato da 1 a q , j da $q + 1$ a $r \Rightarrow r - p + 1$ iterazioni
 - Il ciclo for viene eseguito per $r - p + 1$ volte

- L'equazione di ricorrenza di MERGESORT è quindi

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + f(n) & n > 1 \end{cases}$$

dove $f(n) = \Theta(n)$ è il costo della procedura MERGE

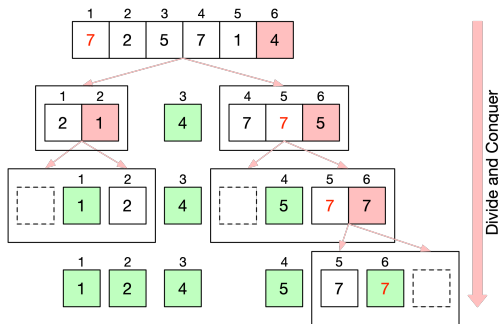
- Risolvendo la ricorrenza con il Master Theorem otteniamo il costo

$$T(n) = \Theta(n \log n)$$

- Il costo di MERGESORT non dipende da come i numeri sono inizialmente organizzati nell'array (caso ottimo, medio e pessimo coincidono)

QUICKSORT (TONY HOARE, 1959)

- 1 Divide:** partizioniamo $A[1, \dots, n]$ in due sotto-array $A_1 = A[1, \dots, q-1]$ e $A_2 = A[q+1, \dots, n]$ tali che tutte le chiavi in A_1 e A_2 siano rispettivamente $\leq$ e $>$ della chiave di $A[q]$ ($A[q]$ scelto durante il partizionamento)
- 2 Conquer:** richiamiamo ricorsivamente l'algoritmo su A_1 e A_2
- 3 Combine:** non necessario ricombinare A_1 e A_2 (sono già ordinati)



Algoritmo non stabile

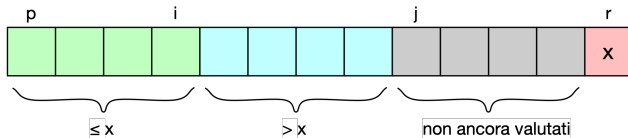
QUICKSORT: PSEUDOCODICE

```
1: function QUICKSORT(INT  $A[1, \dots, n]$ , INT  $p$ , INT  $r$ )
2:   if  $p < r$  then
3:      $q = \text{PARTITION}(A, p, r)$ 
4:      $\triangleright$  Here  $\forall i \leq q-1, A[i] \leq A[q]$  and  $\forall i \geq q+1, A[i] > A[q]$ 
5:     QUICKSORT( $A, p, q-1$ )
6:     QUICKSORT( $A, q+1, r$ )
```

- Prima chiamata: $\text{QUICKSORT}(A, 1, n)$
- Partizionamento complesso ma non necessario ricombinare i sotto-array
- La funzione `PARTITION` sceglie un elemento **pivot** e riorganizza il sotto-array $A[p, \dots, r]$ in modo che
 - Tutti le chiavi più piccole della chiave pivot sono nella parte sinistra
 - Tutti le chiavi più grandi della chiave pivot sono nella parte destra
 - Ritorna l'indice q della posizione del pivot in $A[p, \dots, r]$

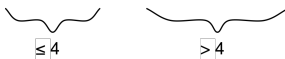
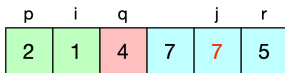
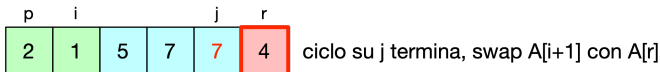
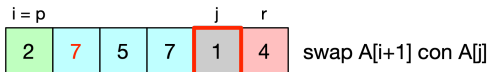
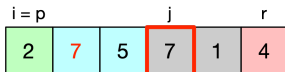
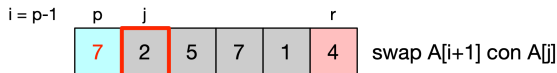
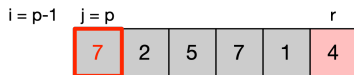
QUICKSORT: PSEUDOCODICE DI PARTITION

```
1: function PARTITION(INT  $A[1, \dots, n]$ , INT  $p$ , INT  $r$ )  $\rightarrow$  INT  
2:    $x = A[r]$   $\triangleright$  Deterministic choice of the pivot  
3:    $i = p - 1$   
4:   for  $j = p, \dots, r - 1$  do  
5:     if  $A[j] \leq x$  then  
6:       SWAP( $A, i + 1, j$ )  
7:        $i = i + 1$   
8:   SWAP( $A, i + 1, r$ )  $\triangleright$  Move the pivot value in  $A[i + 1]$   
9:   return  $i + 1$   $\triangleright$  Return the pivot index in  $A$ 
```



- Costo nel caso ottimo, medio e pessimo: $\Theta(r - p + 1)$
 - PARTITION *valuta* ogni cella in $A[p, \dots, r]$
- PARTITION agisce in-place ma non è stabile

QUICKSORT: ESECUZIONE DI PARTITION



COMPLESSITÀ COMPUTAZIONALE DI QUICKSORT

- Il costo computazionale di QUICKSORT dipende da quanto bene PARTITION riesce a partizionare l'array A nei due sotto-array

$$A[p, \dots, q - 1] \text{ and } A[q + 1, \dots, r]$$

- La scelta del pivot comporta sotto-array **bilanciati** o **sbilanciati**
 - Sono bilanciati se hanno approssimativamente la stessa lunghezza
 - Sono sbilanciati se le loro lunghezze differiscono di molto
- Il partizionamento determina i costi nel casi ottimo, medio e pessimo

ANALISI DEL CASO PESSIMO DI QUICKSORT

- **Partizionamento pessimo:** un array ha lunghezza 0 e l'altro $n - 1$

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + T(0) + n & n > 1 \end{cases}$$

- Possiamo risolvere l'equazione di ricorrenza col metodo iterativo

$$\begin{aligned} T(n) &= T(n-1) + T(0) + n \\ &= T(n-1) + 1 + n \\ &= T(n-2) + 2 + (n-1) + n \\ &= T(n-3) + 3 + (n-2) + (n-1) + n \\ &\dots \end{aligned}$$

$$= T(n-i) + i + \sum_{k=0}^{i-1} n - k$$

La ricorsione termina per $n - i = 0 \Rightarrow i = n$. Il costo nel **caso pessimo** è

$$T(n) = 1 + n + \sum_{k=0}^{n-1} (n-k) = 1 + n + \sum_{k=1}^n k = 1 + n + \frac{n(n+1)}{2} = \Theta(n^2)$$

ANALISI DEL CASO OTTIMO DI QUICKSORT

- **Partizionamento ottimo**: i due sotto-array sono entrambi lunghi $n/2$

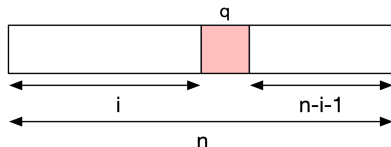
$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

- Possiamo usare il Master Theorem ($\alpha = \beta = 1$)
- Il costo nel **caso ottimo** è $T(n) = \Theta(n \log n)$
- Nel caso ottimo stesso costo di MERGESORT
- Nel caso pessimo, stesso costo di INSERTIONSORT and SELECTIONSORT

ANALISI DEL CASO MEDIO DI QUICKSORT

- Nel caso generale, l'equazione di ricorrenza di QUICKSORT è

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(i) + T(n-i-1) + n & n > 1 \end{cases}$$



- Problema: i (e $n-i-1$) possono cambiare in ogni chiamata ricorsiva
- Sotto l'assunzione probabilistica che tutte le partizioni siano **equiprobabili**, possiamo considerare il valore medio e valutare l'equazione di ricorrenza

$$T(n) = \begin{cases} 1 & n \leq 1 \\ \frac{1}{n} \sum_{i=0}^{n-1} [T(i) + T(n-i-1)] + n & n > 1 \end{cases}$$

ANALISI DEL CASO MEDIO DI QUICKSORT

- Semplifichiamo l'equazione di ricorrenza del caso medio notando che

$$\sum_{i=0}^{n-1} T(n-i-1) = \sum_{i=0}^{n-1} T(i)$$

ed otteniamo

$$T(n) = \begin{cases} 1 & n \leq 1 \\ \frac{2}{n} \sum_{i=0}^{n-1} T(i) + n & n > 1 \end{cases}$$

- Possiamo dimostrare che tale equazione ha costo $O(n \log n)$

ANALISI DEL CASO MEDIO DI QUICKSORT 1/4

- Applichiamo innanzitutto alcune trasformazioni all'equazione

$$T(n) = \frac{2}{n} \sum_{i=0}^{n-1} T(i) + n$$

- Moltiplicando per n otteniamo

$$nT(n) = 2 \sum_{i=0}^{n-1} T(i) + n^2 \quad (1)$$

- Sostituendo n con $n - 1$ otteniamo

$$(n-1)T(n-1) = 2 \sum_{i=0}^{n-2} T(i) + (n-1)^2 \quad (2)$$

- Sottraendo l'equazione 2 all'equazione 1 otteniamo

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 1$$

- In questo modo siamo riusciti a far sparire la sommatoria

ANALISI DEL CASO MEDIO DI QUICKSORT 2/4

- Riorganizziamo e semplifichiamo l'equazione precedente

$$nT(n) = (n+1)T(n-1) + 2n - 1$$

- Dividendo per $n(n+1)$ otteniamo

$$\begin{aligned}\frac{T(n)}{n+1} &= \frac{T(n-1)}{n} + \frac{2}{n+1} - \frac{1}{n(n+1)} \\ &\leq \frac{T(n-1)}{n} + \frac{2}{n+1}\end{aligned}$$

- Sostituendo n con $n-1$ nella seguente disuguaglianza

$$\frac{T(n)}{n+1} \leq \frac{T(n-1)}{n} + \frac{2}{n+1}$$

abbiamo che

$$\frac{T(n-1)}{n} \leq \frac{T(n-2)}{n-1} + \frac{2}{n}$$

ANALISI DEL CASO MEDIO DI QUICKSORT 3/4

- Applichiamo iterativamente le sostituzioni

$$\begin{aligned}\frac{T(n)}{n+1} &\leq \frac{T(n-1)}{n} + \frac{2}{n+1} \\ &\leq \frac{T(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\ &\leq \frac{T(n-3)}{n-2} + \frac{2}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\ &\dots \\ &\leq \frac{T(n-i-1)}{n-i} + \sum_{k=0}^i \frac{2}{n+1-k}\end{aligned}$$

- La ricorsione termina quando $n - i - 1 = 1 \Rightarrow i = n - 2$

ANALISI DEL CASO MEDIO DI QUICKSORT 4/4

- Sostituendo $i = n - 2$ nella disequazione

$$\frac{T(n)}{n+1} \leq \frac{T(n-i-1)}{n-i} + \sum_{k=0}^i \frac{2}{n+1-k}$$

abbiamo

$$\begin{aligned} \frac{T(n)}{n+1} &\leq \frac{T(1)}{2} + 2 \sum_{k=0}^{n-2} \frac{1}{n+1-k} \\ &= \frac{1}{2} + 2 \sum_{k=3}^{n+1} \frac{1}{k} \\ &\leq \frac{1}{2} + 2 \sum_{k=2}^n \frac{1}{k} \\ &\leq \frac{1}{2} + 2 \int_1^n \frac{1}{k} dk = \frac{1}{2} + 2 \ln n \end{aligned}$$

- Concludiamo quindi che

$$T(n) \leq (n+1) \left(\frac{1}{2} + 2 \ln n \right) = O(n \log n)$$

QUICKSORT: SCELTA DEL PIVOT

- Nella nostra implementazione di PARTITION il pivot è sempre l'ultimo elemento $A[q]$ nel sotto-array $A[p, \dots, q]$
 - Quali sono istanze in input che comportano il caso pessimo?
- La nostra analisi del caso medio si basa sull'assunzione probabilistica che tutte le partizioni siano equiprobabili, mentre con la nostra strategia per la scelta del pivot il bilanciamento delle partizioni dipende dall'ordinamento iniziale del vettore in input
 - Array quasi ordinati comportano il caso pessimo
 - In situazioni reali, array quasi ordinati sono più probabili di array totalmente *disordinati*
- Possiamo utilizzare la randomizzazione per correggere tale problema

```
1: function RPARTITION(INT  $A[1, \dots, n]$ , INT  $p$ , INT  $r$ )  $\rightarrow$  INT  
2:    $i = \text{RANDOM}(p, r)$   $\triangleright$  Random choice of the pivot  
3:   SWAP( $A, i, r$ )  
4:   return PARTITION( $A, p, r$ )
```

- Una scelta randomizzata del pivot aumenta la probabilità che le partizioni siano equiprobabili, così come assunto nell'analisi del caso medio

ALGORITMO DI ORDINAMENTO IN JAVA

- La classe *java.util.Arrays* in Java contiene metodi per manipolare array
- Il metodo *sort* della classe fornisce un algoritmo di ordinamento
- L'algoritmo implementato è una combinazione di InsertionSort, MergeSort e QuickSort
- Il QuickSort implementato è noto con il nome di *DualPivot Quicksort*
 - Utilizza due pivot invece che uno
 - Stesso costo computazionale di QuickSort
 - Costo $O(n \log n)$ su molte istanze su cui QuickSort ha costo pessimo
- InsertionSort è utilizzato su istanze piccole (≤ 47):
 - Più veloce (in sec) di QuickSort e MergeSort
- MergeSort viene preferito a QuickSort su istanze *grandi* (≥ 286) che presentano poche sotto-sequenze decrescenti
 - QuickSort rischia di avere caso pessimo su istanze *quasi ordinate*

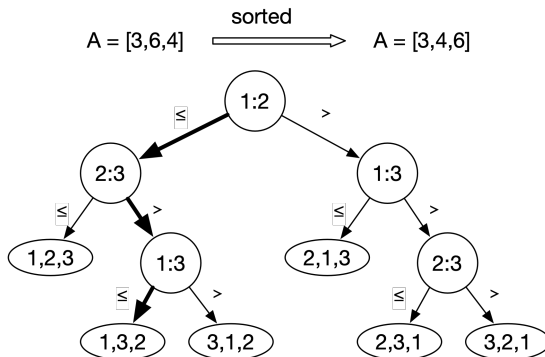
ALGORITMI DI ORDINAMENTO: RIASSUNTO

	Caso ottimo	Caso medio	Caso pessimo
SelectionSort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	$\Theta(n \log n)$	$O(n \log n)$	$\Theta(n^2)$

	In place	Stabile
SelectionSort	Si	No
InsertionSort	Si	Si
MergeSort	No	Si
QuickSort	Si	No

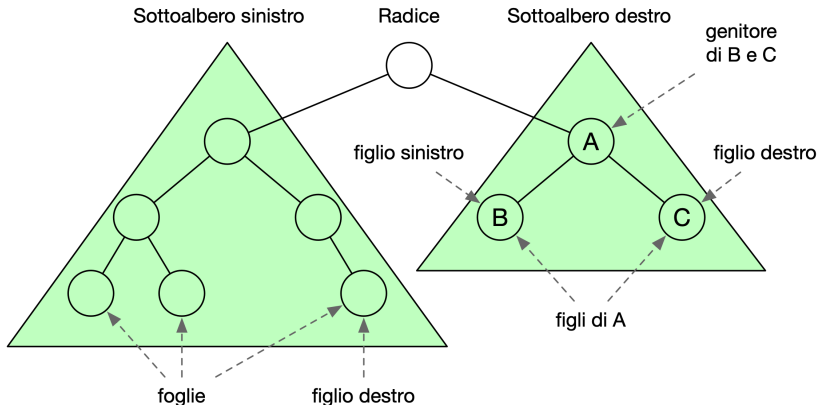
LOWER BOUND PER ORDINAMENTO COMPARATIVO

- Tutti gli algoritmi considerati finora si basano sul confronto
 - L'ordinamento è basato sul confronto ($<$, $=$, $>$) tra valori
- Esistono algoritmi di ordinamento **comparativi** con prestazioni migliori di $O(n \log n)$ nel **caso pessimo**?
- Possiamo descrivere un algoritmo comparativo con un **albero di decisione**
- Esempio: albero di decisione di INSERTIONSORT



ALBERI BINARI: TERMINOLOGIA

- Un **Albero Binario** è un Albero in cui ogni nodo ha **al massimo due figli**



- La **profondità** di un nodo u è la lunghezza del percorso (unico) che va dalla radice al nodo u (numero di archi)
- L'**altezza** di un Albero è la sua massima profondità
- Un Albero con il solo nodo radice ha altezza 0

LOWER BOUND PER ORDINAMENTO COMPARATIVO

■ Idea

- Ogni algoritmo di ordinamento comparativo può essere descritto da un albero di decisione
- Ogni nodo corrisponde al confronto di un paio di elementi nell'array
- L'esecuzione dell'algoritmo può essere tracciata sull'albero seguendo un percorso radice-foglia

■ Proprietà dell'albero di decisione

- Albero binario in cui ogni nodo interno ha **esattamente** due figli
- La sua **altezza** corrisponde al numero di confronti nel **caso pessimo**
- L'altezza media corrisponde al numero di confronti nel caso medio
- Il **numero di foglie** dell'albero è almeno uguale a **$n!$** , i.e. il numero possibile di permutazioni degli indici di un array di lunghezza n

LOWER BOUND PER ORDINAMENTO COMPARATIVO

Teorema

Sia T_k un albero binario con k foglie in cui ogni nodo interno ha esattamente due figli e sia $h(T_k)$ l'altezza di T_k . Allora $h(T_k) \geq \log_2 k$

- Dimostrazione (per induzione su k)
 - Caso base: $h(T_1) = 0 \geq \log_2 1 = 0$
 - Induzione: siano T_{k_1} e T_{k_2} rispettivamente il sotto-albero sinistro e destro di T_k , dove $k = k_1 + k_2$ e assumiamo che $k_1 \geq k_2$. La nostra ipotesi induttiva è che $h(T_{k'}) \geq \log_2 k'$ per ogni $k' < k$. Allora

$$\begin{aligned}h(T_k) &= 1 + \max(h(T_{k_1}), h(T_{k_2})) \\&\geq 1 + h(T_{k_1}) \\&\geq 1 + \log_2 k_1 && \text{(per ipotesi induttiva)} \\&= \log_2 2 + \log_2 k_1 = \log_2 2k_1 && \text{(abbiamo assunto } k_1 \geq k_2\text{)} \\&\geq \log_2 (k_1 + k_2) = \log_2 k\end{aligned}$$

LOWER BOUND PER ORDINAMENTO COMPARATIVO

Teorema

Ogni algoritmo di ordinamento comparativo richiede $\Omega(n \log n)$ confronti nel caso pessimo

■ Dimostrazione

- Ogni algoritmo di ordinamento comparativo richiede nel caso pessimo un tempo di calcolo proporzionale all'altezza dell'albero di decisione (numero di confronti nel caso pessimo)
- L'albero di decisione ha almeno $n!$ foglie, dove n è il numero di elementi da ordinare
- Per il teorema precedente, l'altezza di un albero binario in cui ogni nodo interno ha esattamente due figli e con almeno $n!$ foglie è

$$\Omega(\log n!) = \Omega(n \log n)$$

- Conseguenza: MERGESORT è un algoritmo di ordinamento comparativo **asintoticamente ottimo**

ORDINAMENTO IN TEMPO LINEARE

- Il lower bound per il caso pessimo vale solo per algoritmi comparativi
- Possiamo utilizzare altre operazioni per determinare un ordinamento
 - Il lower-bound per il caso pessimo non si applica a tali algoritmi
 - Questo tipicamente implica vincoli sul tipo di elementi da ordinare
- Vediamo due algoritmi di ordinamento che, sotto determinate condizioni, possono ordinare in tempo lineare nel caso pessimo
 - CountingSort
 - RadixSort

COUNTINGSORT (HAROLD H. SEWARD, 1954)

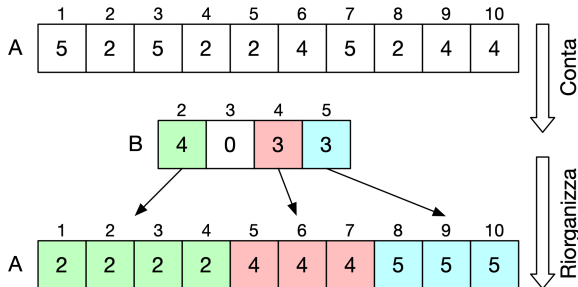
■ **Assunzioni:** le chiavi da ordinare sono **interi nell'intervallo $[a, b]$**

■ a e b possono essere interi negativi

■ **Idea del CountingSort:**

1 Usa un array $B[]$ di lunghezza $b - a + 1$ per **contare** quante volte ogni valore chiave $x \in [a, b]$ appare nell'array in input $A[]$

2 **Riorganizza** tali valori in $A[]$



Nota: nell'esempio usiamo un array $B[a, \dots, b]$ per semplicità

COUNTINGSORT: PSEUDOCODICE

```
1: function COUNTINGSORT(INT  $A[1, \dots, n]$ )
2:    $a = \text{MIN}(A)$ ,  $b = \text{MAX}(A)$ ,  $k = b - a + 1$ 
3:   LET  $B[1, \dots, k]$  BE A NEW ARRAY
4:   for  $i = 1, \dots, k$  do
5:      $B[i] = 0$ 
6:   ▷ Count
7:   for  $i = 1, \dots, n$  do
8:      $B[A[i] - a + 1] = B[A[i] - a + 1] + 1$ 
9:   ▷ Re-arrange
10:   $j = 1$ 
11:  for  $i = 1, \dots, k$  do
12:    while  $B[i] > 0$  do
13:       $A[j] = i + a - 1$ 
14:       $B[i] = B[i] - 1$ 
15:       $j = j + 1$ 
```

- Usiamo un array $B[1, \dots, b - a + 1]$ e riscaliamo da $[a, b]$ in $[1, b - a + 1]$
 - per ogni $x \in [a, b]$, $x - a + 1 \in [1, b - a + 1]$
 - per ogni $x \in [1, b - a + 1]$, $x + a - 1 \in [a, b]$

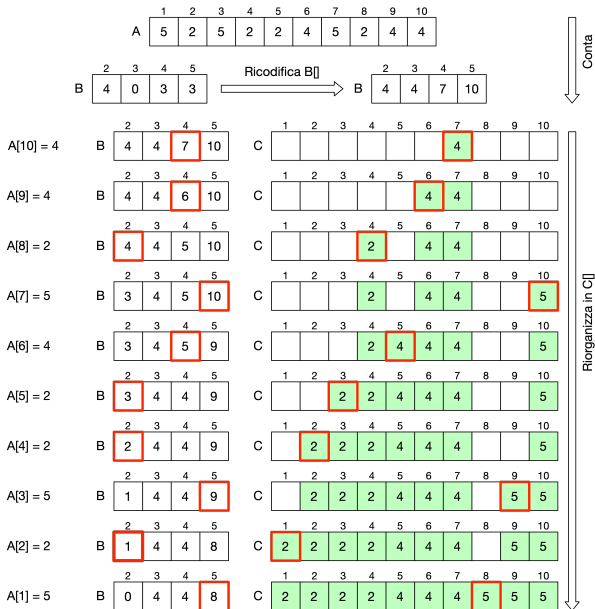
COMPLESSITÀ COMPUTAZIONALE DI COUNTINGSORT

- Costo pessimo, medio e ottimo: $\Theta(n + k) = \Theta(\max(n, k))$, $k = b - a + 1$
 - Le funzioni MIN e MAX costano $\Theta(n)$
 - Il ciclo for a riga 4 viene eseguito k volte
 - Il ciclo for a riga 7 viene eseguito n volte
 - Il costo dei cicli for e while a riga 11 e 12 è $n + k$
 - Il ciclo for viene eseguito k volte
 - La somma totale degli interi in B è uguale ad n
- Il costo dipende anche dalla dimensione del range $[a, b]$
 - Il range $k = b - a + 1$ potrebbe essere molto più grande di n
 - Ex. Se $A = [1, 2^{100}]$ allora $n = 2$, $k = 2^{100}$
- Se $k = O(n)$ allora il costo del CountingSort è $\Theta(n)$

ULTERIORI CONSIDERAZIONI SU COUNTINGSORT

- La nostra implementazione di CountingSort ordina solo array di interi
- Non possiamo usare tale implementazione per ordinare array di **dati** associati a **chiavi intere**
- Comunque, possiamo modificare di poco la nostra implementazione per ottenere un algoritmo di ordinamento lineare più generale
- La nostra nuova implementazione può essere anche stabile
- Benchè aggiungiamo un po' di overhead in termini di memoria e tempo di calcolo il costo asintotico in termini di memoria e tempo rimane invariato

COUNTINGSORT (CHIAVI E DATI): ESEMPIO



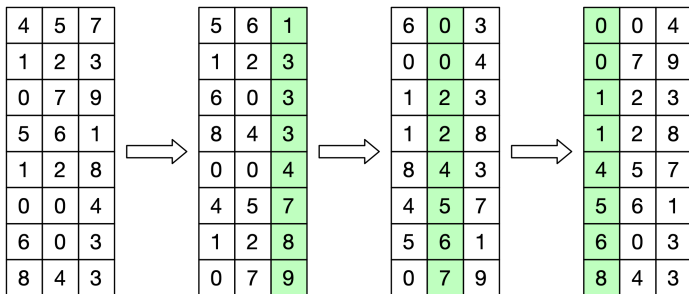
COUNTINGSORT (CHIAVI E DATI): PSEUDOCODICE

```
1: function COUNTINGSORT(INT  $A[1, \dots, n]$ )
2:    $a = \text{MINKEY}(A)$ ,  $b = \text{MAXKEY}(A)$ ,  $k = b - a + 1$ 
3:   LET  $B[1, \dots, k]$  BE A NEW ARRAY
4:   LET  $C[1, \dots, n]$  BE A NEW ARRAY
5:   for  $i = 1, \dots, k$  do  $B[i] = 0$ 
6:    $\triangleright$  Count
7:   for  $i = 1, \dots, n$  do  $B[A[i].\text{key} - a + 1] = B[A[i].\text{key} - a + 1] + 1$ 
8:    $\triangleright$  Recode  $B[]$ 
9:   for  $i = 2, \dots, k$  do
10:     $B[i] = B[i] + B[i - 1]$   $\triangleright B[i]$  now contains the number of keys  $\leq i + a - 1$ 
11:    $\triangleright$  Re-arrange in  $C[]$ 
12:   for  $i = n, \dots, 1$  do
13:     $C[B[A[i].\text{key} - a + 1]] = A[i]$   $\triangleright$  Copy keys and data from  $A[]$  to  $C[]$ 
14:     $B[A[i].\text{key} - a + 1] = B[A[i].\text{key} - a + 1] - 1$ 
15:   for  $i = 1, \dots, n$  do
16:     $A[i] = C[i]$   $\triangleright$  Copy sorted data from  $C[]$  to  $A[]$ 
```

- Il ciclo da n ad 1 a riga 12 rende la nuova implementazione stabile

RADIXSORT (HAROLD H. SEWARD, 1954)

- **Assunzioni:** i valori chiave sono composti da cifre o caratteri
 - La radice è il numero di cifre univoche che rappresentano un numero
- **Idea del RadixSort:**
 - 1 Ordina prima rispetto alla cifra meno significativa, poi rispetto alla penultima cifra meno significativa e così via
 - 2 L'ordinamento delle cifre deve essere stabile



RADIXSORT CON ORDINAMENTO NON STABILE

4	5	7
1	2	3
0	7	9
5	6	1
1	2	8
0	0	4
6	0	3
8	4	3



5	6	1
1	2	3
6	0	3
8	4	3
0	0	4
4	5	7
1	2	8
0	7	9



6	0	3
0	0	4
1	2	8
1	2	3
8	4	3
4	5	7
5	6	1
0	7	9



0	0	4
0	7	9
1	2	8
1	2	3
4	5	7
5	6	1
6	0	3
8	4	3

RADIXSORT: PSEUDOCODICE E COMPLESSITÀ

```
1: function RADIXSORT(INT  $A[1, \dots, n]$ )  
2:    $d = \text{max key length}$   
3:   for  $i = 1, \dots, d$  do  
4:     stable sort of A on the i-th digit of the keys
```

- Il massimo numero di cifre in una chiave è d
- Le cifre sono numerate da 1 a d da destra verso sinistra
- Se usiamo COUNTINGSORT come algoritmo stabile il costo ottimo, medio e pessimo di RADIXSORT è $\Theta(d(n + k))$
 - d è il numero massimo di cifre in una chiave
 - k è il numero di possibili valori per una cifra
 - Se le chiavi sono interi $k = 10$, mentre se le chiavi sono stringhe $k = \text{numero di caratteri ammessi nella stringa}$
- Se $k = O(n)$ e d è un valore costante (lunghezza massima limitata) allora il costo è $\Theta(n)$ (lineare sul numero di elementi in $A[]$)

RIASSUNTO

	Caso ottimo	Caso medio	Caso pessimo
SelectionSort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
InsertionSort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
QuickSort	$\Theta(n \log n)$	$O(n \log n)$	$\Theta(n^2)$
CountingSort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$
RadixSort	$\Theta(d(n + k))$	$\Theta(d(n + k))$	$\Theta(d(n + k))$

	In place	Stabile
SelectionSort	Si	No
InsertionSort	Si	Si
MergeSort	No	Si
QuickSort	Si	No
CountingSort	No	Si
RadixSort	No	Si

- n = numero di elementi da ordinare
- d = numero massimo di cifre in una chiave
- k = ampiezza del range di valori chiave